

SRW3 Phase 1H-R1 — Lineage Idempotence and SecurityContext Binding Repair Report

Designation	PHASE 1H-R1 COMPLETE — implementation defects repaired; real-client boundary result preserved
Repair branch	phase1h-r1-fix (created from phase1h @ 4cb9ab7be39e8d88945d8d6ea7876b819b7b87ee, tag phase1h-complete)
Type	Repair and closure pass — NOT a new research phase
Scope	Two concrete defects found by independent review of Phase 1H: (1) lineage replay-idempotence not actually implemented; (2) SecurityContext_H constructed but never supplied to or consumed by GateH.evaluate()
Client	Geth 1.17.7-stable @ 3d858f858a458effb2a563788aedef1fe65e1f0d3 (unmodified, re-provisioned from the pinned gethstore tarball; commit identical to the Phase 1H provenance record)
Verification	Live re-run of the complete Phase 1H suite against a real Geth devnet driven through the Engine API, plus two new dedicated R1 test suites (37 unit checks) and a source-level call-site audit

1. Executive summary

Independent review of the completed Phase 1H checkpoint identified two implementation-level defects. First, the lineage store persisted the wall-clock recording time inside the record and then compared entire JSON objects for equality, so an otherwise identical replay of the same security-semantic record could be falsely flagged as a lineage inconsistency — the replay-idempotence property claimed by Phase 1H was not genuinely implemented. Second, the harness constructed a SecurityContext_H object for every evaluated payload and then discarded it: the gate evaluated purely from local policy/evidence state plus an optional context_override, meaning the security context was never a real input to the gate decision.

Both defects were repaired. The repair is deliberately narrow: no new research claims, no new theorem families, no client or protocol changes. The corrected system was then verified on three independent axes. (1) Two new dedicated suites — LID (11 checks) for lineage semantic idempotence and SCT/AUTH/DET (26 checks) for SecurityContext binding, mutation rejection, authority preservation, and digest determinism — pass fully. (2) The complete Phase 1H experimental surface was re-executed live against a fresh Geth 1.17.7 devnet: H1-H10, H-A1-H-A16, the consensus-visible simulation, BAL analysis, determinism, restart/replay, duplicate-payload handling, and performance. Every verdict and every layer matches the original phase1h recording exactly (zero regression); the strengthened restart test now additionally demonstrates byte-level lineage stability and security-context digest stability across a real client restart. (3) A source-level audit of every call site of GateH.evaluate(), LineageStore.record(), and build_security_context() confirms there is no path in which the security context is constructed and discarded, the gate silently rebuilds a replacement context, a replay timestamp changes semantic equality, a duplicate record overwrites an existing one, or security-context data bypasses the authority checks.

The scientific designation of the checkpoint is updated, not upgraded:

Phase 1H real-client boundary integration, with corrected lineage semantics and first-class security-context binding.

Nothing in this repair changes the Phase 1H boundary conclusions. The repair does not modify Ethereum consensus, does not modify Geth, does not prove protocol-level enforcement, and does

not close A-G2 or A-G7.

2. Starting state and historical preservation

The repair started from phase1h @ 4cb9ab7be39e8d88945d8d6ea7876b819b7b87ee (tag phase1h-complete) as the sole starting point, with the new branch phase1h-r1-fix created from it. The frozen Phase 1G material was verified unchanged before any edit, using the prescribed commands (git status, git log --oneline --decorate -10, git diff phase1g..phase1h -- k/ phase1e/ phase1f/ phase1g/ — the diff is empty, exit 0). The frozen milestone branches phase1d-r1-complete @ 668a651, phase1e @ 8b26cb7, phase1f @ b6c651e, phase1g @ a71ba64, and phase1h @ 4cb9ab7 are not rewritten, amended, or force-pushed at any point in this pass; the repair work lives either under additive phase1h-r1-fix/ paths or as necessary modifications to existing Phase 1H Python implementation files, which the mandate explicitly permits.

The environment was re-provisioned from the Phase 1H pinned recipe: the Geth binary was re-downloaded from the exact recorded tarball (geth-linux-amd64-1.17.7-3d858f85.tar.gz) and its build commit verified against the Phase 1H provenance entry (3d858f858a458effb2a563788aedf1fe65e1f0d3), so every live re-run in this report executed against the identical unmodified client. The K toolchain remains unavailable in this environment, exactly as disclosed in Phase 1H; this repair is therefore classified as an implementation-level correctness repair (see section 7).

Recorded identifiers (full values in phase1h-r1-fix/provenance/):

Item	Value
Base phase1h SHA	4cb9ab7be39e8d88945d8d6ea7876b819b7b87ee
Repair branch	phase1h-r1-fix
Final repair commit SHA	recorded in provenance/PROVENANCE.md (this document is part of that commit)
Changed implementation files	12 modified + 5 new, hashes in provenance/provenance-hashes.txt

3. Defect 1 — lineage replay-idempotence (R1)

3.1 Original bug

Phase 1H's LineageStore.record() built the record as a single JSON object in which the wall-clock recording time was a first-class field:

```
rec = { "blockHash": ..., "verdict": ..., ..., "recordedAt": time.time() }
...
if old == rec: # full-object equality
    return rec # replay-idempotent
raise RuntimeError("SRW3 lineage inconsistency: ...")
```

Because recordedAt was inside the persisted object and the equality test compared the entire object, two recordings of the same security-semantic content could never compare equal: time.time() advances between calls, so the second comparison failed and the store raised the lineage-inconsistency error for an honest replay. The docstring claimed replay-idempotence; the implementation did not have it. The defect was latent in the committed Phase 1H suite because the existing tests never re-invoked record() with identical semantics through the harness path — H-A15, for example, verified duplicate client newPayload submissions but only counted records, never re-recorded. A replay after a client restart (or any duplicate harness processing of an already-recorded payload) would have hit the false inconsistency.

A second, compounding factor: the harness advanced the lineage head (`set_head(block_hash)`) after every recording, so a duplicate processing of an older payload would also have derived a different `lineageHeadBefore` than the original record — an additional non-timestamp source of the same false inconsistency. Both sources had to be addressed for replay-idempotence to be genuinely true.

3.2 Repair

The store now distinguishes **security-semantic record fields** — the fixed identity fields (`blockHash`, `parentHash`, `verdict`, `evidenceDigest`, `authorityCertificate`, `policyVersion`, `executionId`, `lineageHeadBefore`) plus every security-relevant extra field (`srw3Layer`, `reason`, `slot`, and now `securityContextDigest`) — from **non-semantic observation metadata** (`recordedAt` and any future `obs_*` / `session_*` process metadata, classified by `is_observation_key()`). The canonical semantic projection is computed by a dedicated function:

```
def semantic_record(record: dict) -> dict:
    return {k: v for k, v in record.items() if not is_observation_key(k)}
```

When the record file already exists, `record()` loads the old record, compares semantic projections (never raw objects), and applies exactly two rules. If the projections are equal, the call is an honest replay: the store returns the **previously persisted record** (including its original `recordedAt`), does not rewrite the file, and leaves the persisted bytes untouched — the comparison is on JSON-loaded structures, so file formatting never matters. If the projections differ in **any** key, the store raises a deterministic `LineageInconsistencyError` (a `RuntimeError` subclass, preserving Phase 1H handler compatibility) whose message names the sorted list of differing semantic fields. Differences are never ignored: observation metadata is excluded from the projection entirely, so it can neither trigger a false inconsistency nor mask a semantic mutation.

The harness side was repaired to match. `SRW3Harness.process()` now checks for an existing record for the payload before evaluation and, when one exists (an authorized replay), reconstructs the evaluation's applicable lineage head **from the persisted record's** `lineageHeadBefore` rather than from the advanced store head, and re-records with that same value. This is the section-14 restart/replay recipe, applied uniformly to any duplicate processing; it closes both the timestamp source and the head-advance source of false inconsistency.

3.3 Duplicate semantics after the repair

Second <code>record()</code> for the same <code>blockHash</code>	Behavior
Semantically identical (any <code>recordedAt</code>)	IDEMPOTENT — returns the persisted record; file bytes unchanged; no second record; no error
Different verdict / evidence digest / authority certificate / policy version / execution id / lineage head	FAIL-CLOSED — <code>LineageInconsistencyError</code> naming the differing fields
Different security-relevant extra (<code>srw3Layer</code> , <code>reason</code> , <code>slot</code> , <code>securityContextDigest</code> , ...)	FAIL-CLOSED — same error class
Only observation metadata differs (<code>recordedAt</code> , <code>obs_*</code> , <code>session_*</code>)	IDEMPOTENT — metadata is outside the semantic identity

3.4 Hostile mutation behavior

A mutated record is never accepted silently. Verdict mutation (LID-3), evidence-digest mutation (LID-4), authority-certificate mutation (LID-5), policy-version mutation (LID-6), and security-context-digest mutation (LID-7) all fail closed at the store layer, independent of anything the gate does. Two supplementary cases strengthen the guarantee: mutation of any semantic extra field such as `slot` also fails (LID-8), and an observation-metadata difference on top of a semantic mutation does not weaken the rejection — metadata cannot mask a semantic difference (LID-9). The inconsistency message is deterministic across repeated attempts (LID-MSG), naming the offending

fields, so the failure is auditable.

3.5 Test results (LID suite — `test_lineage_r1.py`)

Test	Requirement	Observed	Result
LID-1	exact duplicate replay, independently generated timestamps	same persisted record returned; recordedAt unchanged; file bytes unchanged; 1 record	PASS
LID-2	duplicate differing only in recordedAt	IDEMPOTENT (semantic projection ignores recordedAt)	PASS
LID-3	duplicate with different verdict	REJECT / lineage inconsistency	PASS
LID-4	duplicate with different evidence digest	REJECT / lineage inconsistency	PASS
LID-5	duplicate with different authority certificate	REJECT / lineage inconsistency	PASS
LID-6	duplicate with different policy version	REJECT / lineage inconsistency	PASS
LID-7	duplicate with different security-context digest	REJECT / lineage inconsistency	PASS
LID-8	duplicate with mutated semantic extra (slot)	REJECT / lineage inconsistency	PASS
LID-9	metadata difference cannot mask verdict mutation	REJECT / lineage inconsistency	PASS
LID-10	restart: new store instance over the same directory, replay	same record, bytes unchanged, no duplicate, head preserved	PASS
LID-M SG	inconsistency message determinism	identical message across attempts, names differing field	PASS

LID suite: 11/11 PASS. Transcript: `phase1h-r1-fix/transcripts/repair/lineage_idempotence.json`.

4. Defect 2 — first-class SecurityContext binding (R2)

4.1 Original issue

Phase 1H's harness constructed a security context for every processed payload and then dropped it:

```
sc = self._security_context() # constructed ...
verdict, checks = self.gate.evaluate( # ... and never passed
ev, presented_effects=..., context_override=..., skip_invariants=...)
```

The gate therefore evaluated from its own local policy/evidence state plus an optional `context_override` dict — a channel that mixes harness-side expectation pins with attacker-presented values. Whatever the gate's individual checks achieved, the constructed context object was not part of the gate decision: there was no validation that the context built for this payload is the context the gate reasons about, no integrity check on the context, and no binding of the context into the lineage record. That is not a `SecurityContext` binding in any meaningful sense, and the two objects the context should have tied together — the deployment-authorized policy material and the client-derived execution evidence — were only implicitly related through the gate's internals.

4.2 New API

GateH.evaluate() now takes the security context as a **required, first-class input**:

```
verdict, checks = gate.evaluate(
    ev,
    security_context=sc, # authoritative/evaluated context (required)
    presented_effects=presented_effects,
    presented_context=presented_context, # attacker-presented/substituted bindings
    expectations=expectations, # evaluation-time pins (expectedParentRoot, ...)
    skip_invariants=skip_invariants,
    lineage_head_expectation=..., # authorized-replay lineage head (from the record)
)
```

The naming separates the three roles that the old `context_override` conflated. `security_context` is the authoritative context constructed by the caller for this evaluation; `presented_context` carries attacker-presented/substituted bindings (`presentedPolicyVersion`, `presentedConfigDigest`, `presentedChainId`, `presentedLineageHead`, `authorizedClientSubject`) that the gate checks against authoritative state at the L3/L4 layers; `expectations` carries the stale-evidence pins (`expectedParentRoot`, `expectedChildRoot`, `expectedLineageHead`). Because `security_context` has no default, any call site that does not supply one fails immediately with a `TypeError` — the API cannot silently operate without a context, and the old-style calls could not survive the repair unnoticed.

The division of labor is architectural: the **caller (harness/runner) constructs** the context from deterministic inputs; the **gate consumes and validates** it. The gate contains no context-construction code path at all (verified programmatically, AUTH-5), and there is no fallback such as `if context_invalid: rebuild_context_from_policy()` anywhere in the gate or harness — an invalid supplied context is a rejection, never a reconstruction (SC-10).

4.3 Canonical context representation

`SecurityContext_H` is now a first-class dataclass with exactly the mandated security-relevant fields as attributes — `policyVersion`, `policyDigest`, `chainId`, `applicationSetDigest`, `interactionGraphDigest`, `lineageHead`, `clientExecutionConfig`, `clientIdentity` — plus `contextDigest` and the field-provenance map (each field records its source, authority, and verification). The previously hidden `_policyDigest` / `_clientIdentity` entries buried in the provenance dict were promoted to real fields.

The context digest is computed over a canonical serialization that is specified, deterministic, and documented in the code:

- domain tag `SRW3-CONTEXT-H-V1`, followed by `k=v` lines in lexicographic key order joined by newlines, keccak-256 hashed;
- `chainId` normalized to its decimal integer string, so `93471` and `"93471"` produce the same digest (DET-3);
- every value is a string produced by `canonical_context_fields()` — no floats, no dict-order dependence, no wall-clock values.

Nondeterministic values are excluded **by construction**: no timestamps, no process ids, no object addresses, no filesystem paths, and no unordered dictionary serialization enter the digest. The digest of an honest context is therefore stable across independent constructions, independent client instances, payload replay, and client restarts (section 6, determinism; and DET-1/DET-2 at unit level).

4.4 The digest is integrity, not authority

The context digest is an integrity and binding mechanism. It is **not** a root of authority, and the gate never treats it as one: every security-relevant field is additionally compared against gate-local authoritative state (SC-1..SC-8), and those comparisons use the gate's own deployment-pinned

policy digest, the client-derived evidence identity/configuration, and the lineage sidecar — never the context's own claims. An attacker who mutates a field and recomputes a valid digest over the mutated context still fails the corresponding field binding (SCT-2b..SCT-10; AUTH-1). This is exactly the section-7 requirement: the attack must fail even when the attacker recomputes the digest.

4.5 Gate integration — SC-1..SC-10

The gate validates the supplied context **before** any substantive invariant/interaction layer runs. A failed SC check returns SRW3_REJECT at the security-context layer with the specific check named, and the substantive layers never execute for an unbound context.

C h e c k	Binding enforced against	Failure reason tag
S C- 1	context.policyVersion == gate-local authorized policy version	SC-1 policy version binding violated
S C- 2	context.policyDigest == deployment-pinned policy digest	SC-2 policy digest binding violated
S C- 3	context.chainId == client-derived evidence chainId	SC-3 chain identity binding violated
S C- 4	context.applicationSetDigest == digest(authorized application set), recomputed	SC-4 application-set binding violated
S C- 5	context.interactionGraphDigest == digest(authorized interaction graph), recomputed	SC-5 interaction-graph binding violated
S C- 6	context.clientExecutionConfig == execution-evidence configuration digest	SC-6 client configuration binding violated
S C- 7	context.clientIdentity == execution-evidence authorized client identity	SC-7 client identity binding violated
S C- 8	context.lineageHead == applicable lineage head (gate's lineage-head source; or the persisted record's head for authorized replay)	SC-8 lineage-head binding violated
S C- 9	recomputed canonical digest == context's recorded digest	SC-9 context digest integrity violated
S C- 1 0	no substitution path exists — architectural	verified by code audit (AUTH-5)

SC-8 deserves a note because it has two legitimate evaluation modes. For a **fresh** payload the gate derives the applicable lineage head through its own `lineage_head_provider` (wired by the harness to the sidecar head, falling back to the client head), so a supplied context carrying a stale or substituted head fails. For an **authorized replay** (a record already exists for the payload) the applicable lineage state is the recorded pre-payload head, and the harness passes exactly that persisted value as `lineage_head_expectation` — the expectation comes from the persisted record, never from a rebuilt context, so the binding remains meaningful.

On success the gate records `checks["SC_security_context"]` with the context digest and binding status, and the harness stores the digest on the processing record (`security_context_digest`) and into

the evidence summary, so every recorded evaluation carries the identity of the context that produced it.

4.6 SecurityContext bound into lineage (section 9)

The lineage record now includes securityContextDigest as a **security-semantic** extra field, giving the binding chain

```
payload → execution evidence → security context → gate verdict → lineage record
```

and making the context digest part of the security-semantic lineage identity for duplicate detection: a replay whose reconstructed context does not reproduce the recorded digest is a genuine semantic disagreement and fails closed (demonstrated live by H-A14's securityContextDigestStable and CS4's contextDigestMatchesRecord). The full mutable context is deliberately not serialized into the record — the digest is the identity.

4.7 Mutation/substitution test results (SCT/AUTH/DET —

test_security_context_r1.py)

The suite uses gate-level fixture evidence (clearly labeled as such; client-derived evidence binding remains established by the live H/H-A suites) and runs each field mutation in two variants: (a) raw mutation with the digest left stale, and (b) mutation **with the digest recomputed** — the digest-must-not-be-authority requirement.

Test	Case	Expected	Observed	Result
SCT-1	honest context	SRW3_VALID	SRW3_VALID, SC checks ok	PASS
SCT-2a/b	mutated policyVersion (raw / digest recomputed)	SRW3_REJECT	SC-1 both variants	PASS
SCT-3a/b	mutated policyDigest	SRW3_REJECT	SC-2 both variants	PASS
SCT-4a/b	mutated applicationSetDigest	SRW3_REJECT	SC-4 both variants	PASS
SCT-5a/b	mutated interactionGraphDigest	SRW3_REJECT	SC-5 both variants	PASS
SCT-6a/b	mutated lineageHead	SRW3_REJECT	SC-8 both variants	PASS
SCT-7a/b	mutated clientExecutionConfig	SRW3_REJECT	SC-6 both variants	PASS
SCT-8a/b	mutated clientIdentity	SRW3_REJECT	SC-7 both variants	PASS
SCT-9	mutated contextDigest, fields intact	SRW3_REJECT	SC-9	PASS
SCT-10	chainId mutated with recomputed digest	SRW3_REJECT	SC-3 (binding, not digest)	PASS

Test	Case	Expected	Observed	Result
AUT H-1	fully attacker-controlled context, self-consistent digest	SRW3_REJECT	SC-1	PASS
AUT H-2	attacker clientIdentity vs client-derived evidence	SRW3_REJECT	SC-7	PASS
AUT H-3	forged protocol-root chain (H-A2 surface) + honest context	forged chain rejected; honest path unaffected	authroot; honest still VALID	PASS
AUT H-4	presented_context substitution (H5 surface)	SRW3_REJECT	authority:authsrc (as in Phase 1H)	PASS
AUT H-5	no construction/substitution path inside the gate	0 gate-side constructions, 1 harness site, no fallback	confirmed	PASS
DET -1	identical inputs → identical digest	equal	equal	PASS
DET -2	no wall-clock/observation contamination	equal across sleep-separated builds; no time-like keys	confirmed	PASS
DET -3	chainId normalization (int vs decimal string)	equal	equal	PASS
DET -4	any field change changes the digest	differs	differs	PASS

SCT/AUTH/DET suite: 26/26 PASS. Transcript:

phaselh-r1-fix/transcripts/repair/security_context_binding.json.

5. Authority semantics preserved

The repair does not touch the Phase 1G authority machinery. The chain remains exactly

```
protocol root → authorized execution client → execution evidence
```

with the 1G NoCircularAuthority checks (no self-citation, strict level decrease, termination at the authorized level-0 root, context-derived per-step authorization) unchanged in `authority.py`. The security context does **not** become an authority source: it does not authorize itself (its every field is compared against gate-local state), its digest is not a root of authority (SCT-10, AUTH-1), the policy does not become self-authorizing (SC-2 compares against the deployment pin held by the gate, not against any policy-carried claim), and the client identity remains context-authorized in the 1G sense (AUTH-2, AUTH-3). The regression evidence is behavioral as well as structural: H5, H-A2, H-A8, and H-A10 — the authority-substitution, forged-root, rogue-client, and circularity surfaces — produce byte-identical verdicts and layers to the Phase 1H recording (section 6).

6. Regression — the complete Phase 1H surface re-run after R1

Method: every Phase 1H runner was executed live against a fresh Geth 1.17.7 devnet (unmodified client, pinned commit) after the repair, and the regenerated transcripts were compared case-by-case against the originals committed at `phaselh` (`git show phaselh:<path>`). The zero-regression criterion is an exact match of the (`srw3 verdict`, `layer`) pair for every case. Every regenerated transcript carries an `_r1` marker recording the repair branch, commit, and `rerunAfterR1: true` — nothing was regenerated silently. The full comparison machine-readable form is `phaselh-r1-fix/transcripts/repair/regression_summary.json` (`zeroRegression: true`).

6.1 H1-H10 (required scenarios)

All ten cases match the original recording exactly: H1 SRW3_VALID; H2 SRW3_REJECT/invariant (aggregate cap on real execution); H3 SRW3_REJECT/effect-completeness (hidden write, presented $\neq$ executed); H4 SRW3_REJECT/evidence-binding (stale parent root); H5 SRW3_REJECT/authority:authsrc; H6/H7 SRW3_REJECT/policy-context; H8 SRW3_REJECT/authcircle; H9 SRW3_REJECT/interaction (oracle write ordered after borrow); H10 DISABLED (non-invasiveness cross-checked by H10-D). The Ethereum-valid / SRW3-invalid separation is intact: every rejected case remains client-VALID.

6.2 H-A1-H-A16 (adversarial)

All sixteen cases match exactly: policy-authority fail-closed (A1), forged root (A2), cap cross-reference (A3), hidden+phantom (A4), stale lineage head (A5), rejected-commitment replay after reorg with no resurrection (A6), policy/config substitution cross-references (A7/A9), rogue client (A8), authority circularity (A10), missing evidence $\rightarrow$ SRW3_ERROR fail-closed (A11), tampered child root (A12), timeout policy (A13), restart (A14), duplicate payload (A15), forkchoice transition (A16).

H-A14 was **strengthened** per the R1 mandate into the full restart/replay recipe: after a real client restart the agent re-read the existing lineage record, reconstructed the same SecurityContext_H from the record's lineageHeadBefore, recomputed the same context digest, replayed the payload through a duplicate engine_newPayloadV5, re-evaluated the same verdict, and called lineage persistence again with the same semantic record. All seven R1 assertions hold: headRestored, recordStable, evidenceDigestStable, securityContextDigestStable, noDuplicateRecord, persistedBytesUnchanged, duplicate status VALID. This closes the original lineage defect on the live path rather than merely testing client duplicate submission.

6.3 Determinism (section 13 extended)

The payload-fixed determinism experiment reproduces all original checks on the repaired system — byte-identical payloads across independent fresh builds (A1 vs A2), identical block hashes across instance A and replay instance B, identical evidence digests, execution ids, and verdicts — and adds the mandated context-determinism checks: securityContextDigestsIdentical_A1_vs_A2 = true and securityContextDigestsIdentical_A_vs_B_replay = true for the compared canonical payloads, across a fresh independent client instance and across replay. No wall-clock value contaminates the context digest. H10-D non-invasiveness also reproduces: chains built with and without the SRW3 layer have identical block hashes. (One implementation note: the replay-side context construction was corrected during R1 to mirror the harness lineage-head rule exactly — shadow-store head precedence — so the comparison is apples-to-apples; the harness itself was not changed for this.)

6.4 Restart, replay, duplicate

Covered live by strengthened H-A14 (restart) and H-A15 (duplicate newPayload — still exactly one lineage record), at unit level by LID-1/LID-10, and on the replay-of-rejected path by H-A6 (deterministic re-rejection, no resurrection) and CS4 (re-presentation of a rejected payload: verdict reproduced, contextDigestMatchesRecord = true, record stable).

6.5 Consensus-visible simulation

CS1-CS5 reproduce the original semantics: honest $\rightarrow$ VALID $\rightarrow$ canonicalized; violating $\rightarrow$ client-VALID but SRW3-REJECT $\rightarrow$ simulated INVALID $\rightarrow$ not canonicalized, head stays at parent; the client still builds and VALIDates a child of the rejected payload (it has no notion of SRW3-invalidity); re-presentation leaves the record stable; recovery after rejection works. The translated-INVALID model, latestValidHash semantics, and fork-choice consequences are unchanged. One benign, explained divergence: CS5's recorded head hash prefix differs from the original because CS3's build-on-rejected step consumed one extra CL build-job timestamp retry in the re-run (the

documented Phase 1H attribute degree of freedom; the drain-settle timing shift moved the race); the verdict (SRW3_VALID) and canonicalization outcome are identical, and CS1/CS2 head hashes match the original byte-for-byte. The divergence is recorded, not silently accepted.

6.6 BAL analysis

Identical results: the BAL write set equals the execution-derived write set (value-equal, with blockAccessIndex attribution); reads remain unattributed keys-only; unchanged invariant bounds are absent from the BAL; the tampered BAL is rejected by the client (INVALID); the gate still requires the ordered trace for the interaction obligation (IO-ORACLE-ORDER reject on the same block). The Phase 1G finding — **BAL $\neq$ complete SRW3 security-effect trace** — is preserved and re-confirmed.

6.7 Performance (re-measured after R1)

Same methodology as Phase 1H (real payloads through the real Engine API; no synthetic microbenchmarks; fresh node per workload class):

Class	txs	Evidence extraction (orig → R1, ms)	Gate time (orig → R1, ms)	Overhead (R1)
small	1	7.6 → 8.14	$\leq 0.13 \rightarrow 0.13$	0.4%
medium	10	7.5 → 11.13	$\leq 0.13 \rightarrow 0.12$	0.5%
large	60	17.5 → 17.13	$\leq 0.13 \rightarrow 0.17$	0.4%

The gate now includes the SC-1..SC-9 validation; it remains below 0.2 ms across all workload classes, and total adapter overhead stays at the Phase 1H envelope (~ 0.4 – 0.5% of produce time). Adapter RSS delta in the re-run: +1.3 MB.

6.8 Environment and toolchain notes

Geth was re-provisioned from the pinned tarball and verified by build commit, so the client is bit-identical to Phase 1H's. Two environment-robustness adjustments were made and are disclosed here: (1) `drain_pool()` now requires two consecutive clean pool samples separated by a settle wait (the re-injection of reverted transactions after a reorg is asynchronous w.r.t. the fork-choice update, and a single clean reading could race it — this is a timing fix with no semantic change); (2) the phase scripts' hardcoded development-checkout paths were replaced by checkout-independent resolution. A frozen-material check confirmed the lineage sidecar directories are runtime state (not committed); the first post-repair run correctly refused to overwrite records left by an intermediate buggy evaluation — the fail-closed store behaving as designed — after which the sidecar was cleared and the canonical re-run recorded above.

7. Formal/K boundary and classification

No K toolchain is available in the repair environment, and no K proof was attempted, mechanized, or claimed. The frozen Phase 1G K machinery and the Phase 1H theorem classification stand exactly as committed. The two repaired properties are classified per the mandate:

- **R1 — semantic lineage idempotence:** implementation-level, experimentally tested (LID suite 11/11; live H-A14 recipe; LID-10 restart persistence). Not a K-proved claim.
- **R2 — first-class SecurityContext consumption/binding:** implementation-level, experimentally tested (SCT/AUTH/DET suite 26/26; live SC layer exercised by every re-run case). Not a K-proved claim.

The digest canonicalization (SRW3-CONTEXT-H-V1) and the semantic-record projection are specified precisely in code precisely so that a future mechanization has a formal target; none is claimed here.

8. Boundary update

The repair does not change the Phase 1H boundary. Explicitly:

- It does **not** change Ethereum consensus — the gate remains shadow/non-consensus; the consensus-visible path remains a CL-side simulation of a hypothetical rule.
- It does **not** modify Geth — the client binary is unmodified (same build commit as Phase 1H); the in-process patch remains unbuilt and disclosed.
- It does **not** prove protocol-level enforcement — consensus-visible SRW3 invalidity still requires a new protocol commitment rule (fork-choice relevance, policy distribution, INVALID-semantics), unchanged.
- It does **not** close A-G2 (protocol root is protocol-given) or A-G7 (policy authority remains a governance input) — the repair deliberately preserves both; the policy remains non-self-authorizing and the context remains a bound input, not an authority.
- It does **not** establish complete execution-trace semantics — the evidence fragment remains the Phase 1H receipt/callTracer/prestateTracer(-diff) + BAL fragment; opcode-level completeness remains open.

The corrected claim wording is: Phase 1H's real-client boundary integration now **genuinely has** the replay-idempotent lineage and the first-class SecurityContext binding that its original report described.

9. Provenance and integrity

Full machine-readable provenance ships in phase1h-r1-fix/provenance/ (PROVENANCE.md, self-excluding MANIFEST.txt, provenance-hashes.txt). Summary:

- Base phase1h @ 4cb9ab7be39e8d88945d8d6ea7876b819b7b87ee; repair branch phase1h-r1-fix; no history rewriting of any milestone branch (verified before and after the work).
- Changed implementation files (12 modified, 5 new) with SHA-256 hashes recorded in provenance-hashes.txt; the repair artifact bundle's final SHA-256 is recorded in PROVENANCE.md and the bundle sidecar.
- Exact test commands and counts: LID 11, SCT/AUTH/DET 26, H-suite 10, adversarial 16, consensus-sim 5, determinism 9 boolean checks, BAL 5, performance 3 classes; toolchain versions recorded (geth 1.17.7 @ 3d858f85, Python 3.12, eth-utils 6.0.0).
- K availability: **no**; nothing was converted into a "PROVED BY K" claim.
- Test counts and the per-case old-vs-new comparison are in transcripts/repair/regression_summary.json; the call-site audit in transcripts/repair/call_site_audit.json.

10. Final scientific status

PHASE 1H-R1 COMPLETE — implementation defects repaired; real-client boundary result preserved.

Demonstrated: real Geth payload execution; Engine API boundary integration; Ethereum-valid vs SRW3-invalid separation; client-derived evidence binding; **corrected lineage replay-idempotence; SecurityContext participation in the gate**; context mutation rejection (including digest-recomputed attacks); deterministic context derivation for identical payloads/contexts across instances, replay and restart; restart/replay semantic stability with byte-identical persistence. Assumed (unchanged): protocol root authority; governance authority over policy; trustworthiness of the local SRW3 implementation itself; software identity beyond what the execution client exposes. Open (unchanged): multi-client consensus interoperability; consensus-authoritative policy distribution; the protocol commitment rule; client/protocol-level consensus enforcement; complete execution-trace semantics; client attestation. Phase 1H has **not** become a consensus-integrated protocol mechanism, and nothing in this repair upgrades it into one.